

实验二：最短路径算法

姓名：郝思粤 学号：PB23151779

2026 年 1 月 24 日

1 实验目的与要求

1. 掌握 Dijkstra 算法的原理及其基于二叉堆（Binary Heap）的优化实现。
2. 能够利用 NetworkX 生成满足特定连通概率的随机图，并进行连通性与负权边检查。
3. 理解最短路径问题的线性规划建模形式，并使用求解器（COPT）进行求解对比。
4. 对不同算法在不同数据规模下的性能进行统计与可视化分析，并给出合理解释。

2 算法设计与实现细节

2.1 随机图生成与预检查

本实验使用 Erdős-Rényi (ER) 模型生成随机图。为了使图在 n 个节点下几乎必然连通，令连边概率满足

$$p > \frac{(1 + \epsilon) \ln n}{n}.$$

代码里取 $\epsilon = 0.5$ ，即 $p = 1.5 \ln(n)/n$ ，并在生成后用 `networkx.is_connected` 二次检查；若不连通则重采样直到连通。边权重在 $[1, 10]$ 内随机生成，保证为正权，从源头上避免 Dijkstra 不可处理的负权边情形。为了增强鲁棒性，仍然实现了统一的合法性检查函数：遍历所有边权，若发现负权立刻报错；同时若图不连通也直接终止。

2.2 Dijkstra (heapq) 实现

Dijkstra 的核心是“每次取当前未确定节点里距离最小者”，朴素做法要 $O(V^2)$ 。本实验用 `heapq` 将“取最小距离点”优化为堆弹出，从而达到 $O(E \log V)$ 。由于 `heapq` 不支持 `decrease-key`，本实现采用“懒惰删除”：每次发现更短距离就直接把新键值压入堆；弹出时如果节点已经在 `visited` 中说明它的最短距离已被确定，直接跳过即可。

2.3 LP 建模与 COPT 求解

最短路径可以视为最小费用流的特例。对有向图 $G = (V, E)$ ，每条边对应变量 $x_{ij} \in [0, 1]$ ，目标最小化总代价 $\sum w_{ij}x_{ij}$ ，并用流量守恒约束保证从源点流出 1 单位流、终点流入 1 单位流、其余点净流量为 0。实验中把无向图通过 `to_directed()` 转为双向边，从而适配该 LP 模型，然后使用 COPT 求解器求解。由于本机未配置 license，COPT 会以非商业限制模式运行，但在本实验规模下仍能正常完成求解与对比。

3 实验结果与分析

3.1 正确性验证

在进行效率统计之前，我先对代码做了一次“可观察”的正确性验证：随机生成一个小规模连通 ER 图 ($n = 12$)，先打印出图的边与对应权重，随后依次验证“图连通”、“边权全为正”、“Dijkstra 输出的路径确实从源点到终点且路径权重和等于输出距离”，最后再与 COPT-LP 的最优目标值进行对比。该验证输出如下（节选）：

- 图边与权重（共 17 条）：

(0, 2, $w = 7$), (0, 5, $w = 4$), (0, 7, $w = 8$), (0, 9, $w = 5$),
(1, 3, $w = 7$), (1, 8, $w = 10$), (1, 9, $w = 3$), (1, 11, $w = 7$),
(2, 9, $w = 8$), (2, 10, $w = 5$), (3, 6, $w = 4$), (4, 5, $w = 8$),
(5, 7, $w = 8$), (5, 8, $w = 3$), (5, 11, $w = 6$), (7, 9, $w = 5$), (10, 11, $w = 2$).

- 图性质检查： $n = 12$ ， $E = 17$ ，连通性为 True；边权范围 $[2, 10]$ ，全部为正；`check_graph_validity(G)` 通过。
- Dijkstra 输出：最短距离 10，路径为 $[0, 5, 11]$ ；路径权重和 $4 + 6 = 10$ 与最短距离一致。
- COPT-LP 输出：最优目标值 10，与 Dijkstra 一致。

这一步的在于，不仅验证了“算法能跑”，还验证了“生成图满足实验要求（连通、正权）”以及“Dijkstra 的路径与距离输出自洽”，并且与 LP 求解结果一致，说明实现与建模均正确。

3.2 代码核心片段与解释

下面摘取代码中两段最核心的实现，说明其作用与设计原因。

(1) Dijkstra 的堆优化与松弛（核心代码）

```
1 priority_queue = [(0.0, source)]
2 visited = set()
3
4 while priority_queue:
5     current_dist, u = heapq.heappop(priority_queue)
6     if u in visited:
7         continue
8     visited.add(u)
9
10    if u == target:
11        break
12
13    for v, edge_data in G[u].items():
14        weight = edge_data.get('weight', 1.0)
15        nd = current_dist + weight
16        if nd < distances[v]:
17            distances[v] = nd
18            predecessors[v] = u
19            heapq.heappush(priority_queue, (nd, v))
```

这段代码完成了 Dijkstra 的主体循环：用小根堆维护“当前已知的最短候选距离”，每次弹出最小的 $(dist, u)$ 并把它加入 `visited`，表示 u 的最短距离已经确定。随后对邻居做松弛（relaxation），若发现更短路径就更新 `distances` 与 `predecessors`，并把新键值压入堆。由于 `heapq` 不支持 `decrease-key`，所以这里采用“懒惰删除”，即允许同一节点出现多次入堆，最终以第一次弹出并进入 `visited` 的那次为准。整体复杂度近似为 $O(E \log V)$ ，很适合本实验这种稀疏随机图场景。

(2) COPT-LP 的流量守恒建模（核心代码）

```
1 for i in G.nodes():
2     b_i = 0.0
3     if i == source:
4         b_i = 1.0
5     elif i == target:
6         b_i = -1.0
7
8     out_edges = list(G.out_edges(i))
9     in_edges = list(G.in_edges(i))
10
11    expr_out = cp.quicksum(x[(i, j)] for (i, j) in out_edges) if
        out_edges else 0.0
```

```

12     expr_in = cp.quicksum(x[(k, i)] for (k, i) in in_edges) if
        in_edges else 0.0
13
14     model.addConstr(expr_out - expr_in == b_i, name=f"flow_{i}")

```

这段代码对应 LP 建模中的“流量守恒约束”。对每个节点 i ，统计其出边变量之和与入边变量之和，并令二者差等于 b_i ：源点 $b_s = 1$ 表示净流出 1，终点 $b_t = -1$ 表示净流入 1，其余节点 $b_i = 0$ 表示中转平衡。把无向图转为有向图后，每条无向边会变成两条方向相反的弧，因此该约束能够表达“从源点到终点走一条路径”的最短路结构。COPT 求解后返回的目标值就是最短路长度（线性松弛下该模型仍会给出整数解，这里与图最短路是一致的）。

3.3 运行时间统计与效率对比

在正确性验证通过后，我对不同规模 $N \in [50, 100, 200, 300, 500]$ 做了性能测试。每个规模生成 10 个随机连通 ER 图样本，统计运行时间均值与标准差（单位：秒）。结果如下表所示，其中 $E(\text{avg})$ 为平均边数；Check 为 Dijkstra 与 COPT-LP 的最短路长度是否一致。

表 1: Dijkstra 与 COPT-LP 求解时间对比（均值 $\pm$ 标准差）

节点数 N	平均边数 $E(\text{avg})$	Dijkstra 时间 (s)	COPT-LP 时间 (s)	Check
50	148.8	0.000096 ± 0.000047	0.002330 ± 0.000369	OK
100	348.5	0.000166 ± 0.000055	0.004237 ± 0.000231	OK
200	803.6	0.000404 ± 0.000121	0.009170 ± 0.000774	OK
300	1247.5	0.000555 ± 0.000221	0.014582 ± 0.004203	OK
500	2292.0	0.000608 ± 0.000426	0.026140 ± 0.005699	OK

3.4 性能趋势图

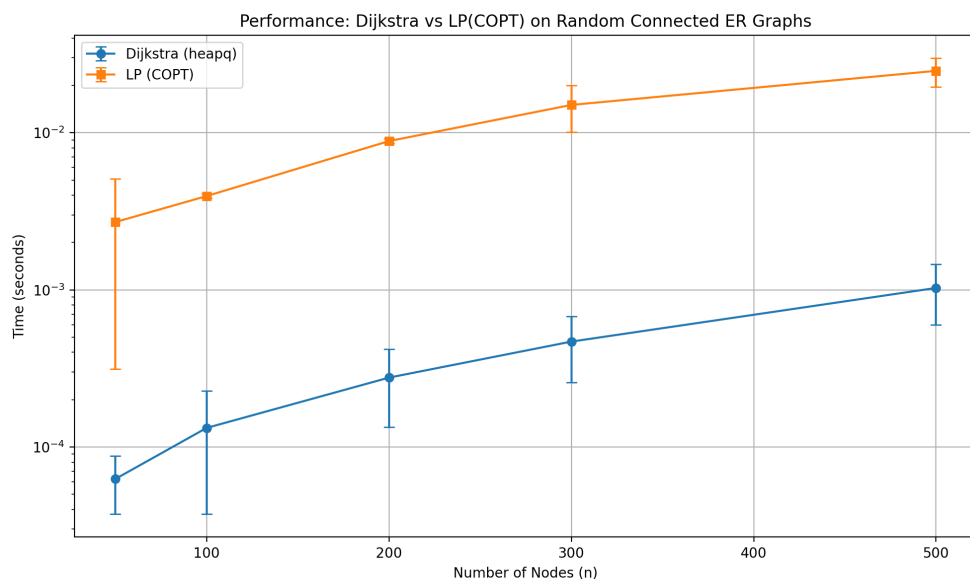


图 1: Dijkstra 与 COPT-LP 运行时间随节点规模变化趋势（纵轴为对数坐标）

3.5 结果分析

从数据上看，两种方法在所有规模下都给出了相同的最短路长度（**Check** 全为 OK），说明实现与建模是正确的。但速度差异非常明显：Dijkstra (heapq) 基本稳定在毫秒以下，而 COPT-LP 随规模增大上升到 10^{-2} 秒量级。原因比较直观：Dijkstra 是为“非负权最短路”专门设计的算法，核心操作是堆的 push/pop，复杂度约为 $O(E \log V)$ ；而 LP 求解器需要先构造变量与约束，再进行通用的预处理和数值计算（例如矩阵分解、迭代求解等），即便最短路问题结构很简单，通用求解仍会产生较大的固定开销和额外的数值开销。因此对“单纯最短路”这种典型图问题，专用算法明显更合适；LP 的优势更多体现在需要叠加复杂线性约束（例如多源多汇、容量约束、额外线性资源限制）时。

4 结论

本实验实现了基于 **heapq** 的 Dijkstra 算法，并加入了连通性与负权边检查，保证随机 ER 图输入下的鲁棒性。实验首先通过小规模图的可观察验证，确认“生成图满足要求、算法输出自洽且与 COPT-LP 一致”；随后在不同规模随机连通图上对比效率，结果表明 Dijkstra 在该任务上显著快于通用的 LP 求解（COPT），且两者结果完全一致。